LLMエージェントによる.NETマルウェアの難読化解除

栗原 北斗[1]　櫻井 慶裕[1]　岡本 剛[1,a)]

概要：.NET Framework で開発されたマルウェアは逆コンパイルが容易であるため，マルウェア開発者は解析を妨害する目的で共通中間言語に難読化を施す．我々の調査では，流行している.NET マルウェアの 52.59%が難読化されていた．本稿では，難読化手法ごとの解除スクリプトを LLM オーケストレーションで生成し，適用後に残った難読化を LLM の推論で解除する 3 段階のフレームワークを提案する．ConfuserEx2 と.NET Reactor で難読化したソース公開マルウェア 5 検体，およびマルウェア開発者が難読化したソース未公開マルウェア 4 検体を対象に評価した．定数とリソースの難読化は解除スクリプトのみで完全に復元できた一方，制御フローの難読化と参照プロキシによる難読化は LLM の推論を要した．いずれの検体からも再コンパイルと再実行が可能な C#コードと実行ファイルを復元でき，解除後のコードからは MITRE ATT&CK Technique ID を全検体で完全に，IoC を F1 で 0.949 以上再現できた．

キーワード：.NET マルウェア，LLM，難読化解除，難読化ツール，マルウェア解析

Deobfuscation of .NET Malware Using LLM-Based Agents

Hokuto KURIHARA[1]　Yoshihiro SAKURAI[1]　Takeshi OKAMOTO[1,a)]

Abstract: Malware written for the .NET Framework is easy to decompile, so malware developers obfuscate its common intermediate language. In our survey, 52.59% of the prevalent .NET malware samples were obfuscated. This paper proposes a three-stage framework that generates a deobfuscation script for each obfuscation technique through LLM orchestration and then leaves the obfuscation remaining after those scripts to LLM reasoning. We evaluated the framework on five open-source malware samples obfuscated with ConfuserEx2 and .NET Reactor, and on four closed-source samples obfuscated by the malware developers. The scripts alone fully recovered constant and resource obfuscation, whereas control-flow obfuscation and reference-proxy obfuscation required LLM reasoning. From every sample, the framework recovered C# code and an executable that could be recompiled and re-executed, and the deobfuscated code reproduced every MITRE ATT&CK Technique ID for all samples and reproduced IoCs with an F1 score of 0.949 or higher.

Keywords: .NET malware, LLM, deobfuscation, obfuscation tool, malware analysis

1. 序論

.NET Framework は，Windows マルウェアの開発基盤としても利用が拡大している．2023 年に UnpacMe へ投稿された 64 ビットの Windows マルウェアのうち，.NET マルウェアが約 25%を占めた [1]．.NET Framework で開発されたプログラムは，共通中間言語（CIL）に加えて，クラス名や変数名などのメタデータを保持する．そのため逆コンパイルが比較的容易であり，悪性プログラムであっても解析しやすい．

この解析のしやすさを打ち消すため，マルウェア開発者は CIL に難読化を施す．我々の調査によると，実際に流行している.NET マルウェアのうち 52.59%が何らかの難読化を施していた．使用された難読化ツールは.NET Reactor 系が 40.63%[2]，ConfuserEx 系が 25.34%[3] であり，この 2 つが上位を占める．

難読化された検体を解析可能な状態へ戻すには，経験と

[1] 神奈川工科大学
Kanagawa Institute of Technology, Atsugi, Kanagawa 243–0292, Japan
[a)] take4@nw.kanagawa-it.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

知識と技術力に加えて時間がかかる．de4dot などのルールベースの手法は，難読化ツールが生成する CIL のパターンに依存する．ツールのアップデートやカスタム改造が施された検体では機能せず，削除された識別子のような意味的な情報も原理的に復元できない．復号メソッドをメモリ上で実行する動的なアプローチは，厳密な隔離環境を要し，制御フローの難読化への対応も限定的である．dnlib や AsmResolver で解析者が解除処理を書く場合も，難読化手法ごとに，また同一ツールでもバージョンが変われば実装をやり直すことになる．いずれも，難読化の変遷に追随する解除ロジックを人手で実装し続ける必要がある．

この追随を人手に頼らない手立てとして，LLM を難読化解除へ適用する研究が始まっている．パターンに依らない制御フローの解除 [4] と，文脈に基づく識別子の意味論的な復元 [5] は一定程度成立した．ルールベースの手法では原理的にできなかった解除や復元が視野に入ったことになる．ただし，探索コストが膨大なもの [6] や，対象が C 言語の小規模なコードに限られるもの [7] にとどまり，実際のマルウェアを解除した例はない．アセンブリや JavaScript でも，複数の手法が組み合わされると解除の難易度が大幅に上昇する [4], [5].

我々の先行研究 [8] でも同じ傾向が現れた．制御フローの難読化に定数の難読化が加わるとビルド成功率が低下する一方で，制御フローと定数の難読化の解除をそれぞれ分離すると成功しやすくなった．ここから得られる見通しは，解除を難読化手法ごとに分け，決定論的に処理できる範囲をスクリプトへ落とすことである．定数（データと文字列を含む）の難読化は復号ルーチンを特定すれば復号でき，制御フローの平坦化も de4dot のように静的に解除できる．いずれも解除の手順が決定論的に定まるため，そのスクリプトの生成自体を LLM に委ねられる．

本研究の目的は，LLM エージェントによって.NET マルウェアの難読化を解除し，マルウェア解析に利用できる C#コードと実行ファイルを復元することである．解除の対象は，階層型難読化 [9] のコード要素レイヤーに属する識別子・制御フロー・定数・参照プロキシ・リソースの 5 手法とする．静的解析そのものを不可能にするパッキングやコード仮想化は，逆コンパイル後の C#に対する構文・意味の復元とは技術基盤が異なるため対象外とする．

提案手法は 3 段階からなる．難読化ツールと難読化手法の組ごとに，役割を分担した複数の LLM エージェントを連携させる LLM オーケストレーションで解除スクリプトを生成し，これを難読化コードへ適用したうえで，残った難読化を LLM エージェントの推論で解除する．スクリプトで解ける箇所を先に処理することで，スクリプトが扱う範囲では解除を決定論的にして再現性を確保しつつ，検体ごとの LLM コストを削減する．スクリプトは同じ組で難読化された他の検体へそのまま適用できるため，検体ごとの生成も不要である．

本研究の貢献は次の 4 点である．

- 代表的な難読化ツール 2 種で 5 つの難読化手法を複合して難読化したソース公開マルウェア 5 検体と，マルウェア開発者が難読化したソース未公開マルウェア 4 検体を対象とし，いずれからも同一の手順で再コンパイルと再実行が可能な C#コードと実行ファイルを復元した．
- 難読化手法ごとに解除スクリプトを自動生成した結果，定数とリソースの難読化はスクリプトのみで完全に復元できる一方，制御フローの難読化と参照プロキシによる難読化はスクリプトでは解除できず，LLM エージェントの推論を要することを明らかにした．
- 解除に成功した試行では，解除後のコードから MITRE ATT&CK Technique ID（以下，ATT&CK）を全検体で完全に，IoC を F1 で 0.949 以上再現でき，1 検体あたり 18 分から 1 時間 38 分で解析可能な状態へ戻せることを示した．
- LLM エージェントの推論は試行ごとに揺らぐが，API 列類似度で解除の成否を判定でき，失敗時に再試行すれば全検体から解除結果を得られた．

2. 難読化技術の体系化と関連研究

2.1 難読化技術の体系化と研究範囲

本研究では，Xu ら [9] が提唱した階層型難読化（Layered Obfuscation）の概念に基づいて難読化技術を 4 つのレイヤーに分類し，このうちコード要素レイヤーを対象とする．本レイヤーには，識別子の難読化，制御フローの難読化（制御フローの平坦化），定数の難読化，および参照プロキシによる難読化（メソッド呼び出しの隠蔽）が含まれる．リソースの難読化はクロスコンポーネントレイヤーに分類されるが，.NET（C#）ではコード要素レイヤーのコードによって復号処理が実行されるため，解除の対象に含める．

一方，ソフトウェアコンポーネントレイヤーとアプリケーションレイヤーは対象外とする．これらに含まれるパッカーやコード仮想化は静的解析そのものを不可能にし，動的なアンパック処理など，逆コンパイル後の C#コードに対する構文・意味の復元とは異なる技術基盤を必要とするためである（付録 A.1.1）.

2.2 既存の難読化解除ツールにおける技術的制約

de4dot やその派生ツールに代表される静的解析アプローチは，特定の難読化ツールが生成する CIL のパターンをシグネチャとして検知し，元の構造に書き換える．しかし，定義されたルールに強く依存するため，難読化ツールのアップデートやカスタム改造が施された検体では解析が困難になる場合が多い．削除された変数名やメソッド名などの意味的な情報を復元することも，パターンマッチングの

© 2026 Information Processing Society of Japan

枠組みでは不可能である．

動的解析を伴うアプローチもある．NETReactorSlayer などは，.NET の Reflection 機能により，暗号化された文字列を復号するメソッドをメモリ上で実行して復元を試みる．ただし，動作を伴うため厳密な隔離環境が求められ，制御フローの難読化を解除する機能も限定的である．

2.3 最新の難読化解除手法と LLM の台頭

Mariano らが提案した Chisel[6] は，実行トレースから制御フローのスケルトンを推論し，プログラム合成を用いて等価コードを再構築する．ルールベースの手法に比べて未知の難読化への耐性が高い一方，探索コストが膨大であり，複雑なロジックを持つマルウェアへの適用には計算リソースの課題が残る．Feng ら [7] は，シンボリック実行で得られた制約条件を LLM に提示することで意味論的な正確性を向上させたが，対象は C 言語の小規模なコードに限定されている．

LLM の解除能力そのものを評価した研究として，アセンブリを対象とした Tkachenko ら [4] と，JavaScript を対象とした Chen ら [5] がある．いずれも，単体の難読化は LLM がある程度解除できる一方，複数の手法が組み合わされるとコードの整合性を維持できず，解除の難易度が大幅に上昇することを示している．

これに対して本研究は，解除精度の向上とマルウェア解析での活用を目的に，解析者が実務で使用する ILSpy などの逆コンパイラが出力する C#コードを対象として，実際のマルウェア検体を用いて解除の到達点を評価する．

3. 提案手法

3.1 手法の全体像

提案手法は，複数の難読化手法が複合的に適用された.NET マルウェアから，解析に使えるコードを復元する．対象とする難読化手法は，2.1 節でコード要素レイヤーに分類した 5 手法である．入力は難読化された実行ファイル，出力は難読化を解除された実行ファイルである．

図 1 に示すとおり，手法は次の 3 段階からなる．

第 1 段階　LLM オーケストレーションにより，難読化手法ごとの解除スクリプトを生成する．ただし，識別子の難読化については解除スクリプトを生成せず，第 3 段階の推論で扱う．

第 2 段階　生成したスクリプトを難読化コードへ適用する．

第 3 段階　第 2 段階で解除できなかった難読化を LLM の推論で解除する．

解除を手法単位に分ける根拠は，先行研究 [8] の結果にある．同研究では，制御フローの難読化に定数の難読化が加わると解除後のビルド成功率が低下する一方で，制御フローと定数の解除をそれぞれ分離すると成功しやすくなることが示された．

段階に分ける利点は 2 つある．第一に，LLM の出力は試行ごとに揺らぐため，委ねる範囲が狭いほど解除結果の再現性が高まる．第二に，スクリプトは難読化ツールと難読化手法の組ごとに一度生成すれば，同じ組で難読化された他の検体にもそのまま適用できるため，検体ごとの生成が不要である．

前提として，解析者の環境で.NET Framework の開発環境，難読化ツール，逆コンパイラ，スクリプト実行環境，および LLM ハーネスが利用できることを想定する．

3.2 第 1 段階：解除スクリプトの生成

図 1 のとおり，メタプロンプトから 8 つのサブエージェントの定義を生成し，最初のサブエージェント（Orchestrator）を起動すると，対象とする 1 つの難読化手法の解除スクリプトが得られる．LLM オーケストレーションの単位は，難読化ツール 1 つと難読化手法 1 つの組である．各サブエージェントの役割と出力は付録 A.2 に示す．

解析者は，難読化ツールの実行ファイルのパスとオプションとともに，次の指示をメタプロンプトへ記述する．

- 検証用サンプルとして無害な.NET Framework のコンソールアプリケーションを 10 個作成し，一つの手法だけでなく複数の手法を組み合わせた難読化を検証すること
- 難読化のオプションはサブエージェント側で選定しないこと
- 解除の対象は CIL ではなく逆コンパイル後の C#とすること
- 復元率・実行結果の一致・可読性の改善などの評価軸を定義し，これらに向けて自ら改善すること
- コードの危険性など人間が判断すべき事項は自ら判断せず，不明点は生成に着手する前に解析者へ質問すること

一つの手法だけでなく複数の手法を組み合わせた難読化を検証する理由は，1 つの手法だけで生成したスクリプトでは解除できないことがあるためである．

3.3 第 2 段階：解除スクリプトの適用

第 2 段階では，難読化された実行ファイルを C#へ逆コンパイルし，生成した解除スクリプトを適用する．

逆コンパイル　図 1 のとおり，改造を施した逆コンパイラ（以下，改造 ILSpy）でクラスごとに C#へ逆コンパイルする．改造の内容は付録 A.1.4 に示す．CIL ではなく C#を対象とするのは，CIL が長大でコンテキスト長に収まらないことと，C#のほうが解析者にとって読みやすいためである．

スクリプトの適用　生成した解除スクリプトをすべて適用する．生成できなかった手法と，適用しても解除できなかった手法は第 3 段階へ回す．

© 2026 Information Processing Society of Japan

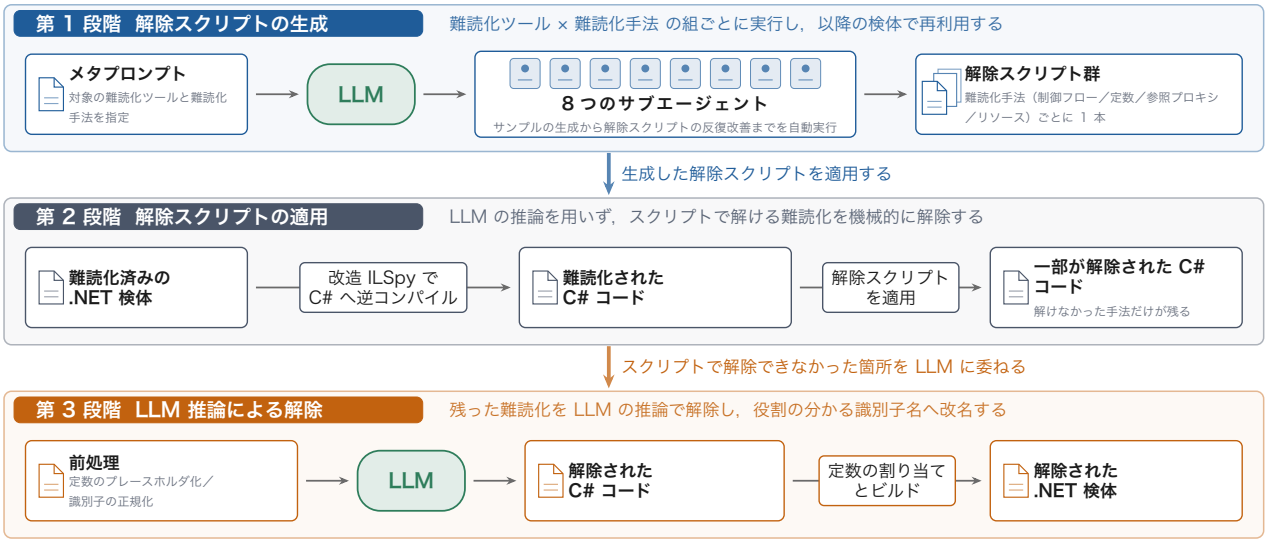


図 1 提案手法の全体像
Fig. 1 Overview of the proposed method.

3.4 第 3 段階：LLM の推論による難読化解除

図 1 のとおり，前処理を施した後に LLM の推論で残った難読化を解除し，復元した定数の値を割り当ててビルドする．割り当てだけは解析者が行い，LLM には行わせない．

3.4.1 前処理：定数のプレースホルダ化

第 2 段階で得られた「一部が解除された C#コード」を複製し，定数の復号データを含む箇所をプレースホルダ（`__CONST_STR_xxx__`）へ置き換える．復号済みの定数を与えると，LLM が学習済みの IoC（ランサムノートの本文，C2 サーバの URL，検体名など）を手がかりに推論して，解除性能を正しく評価できなくなるためである．

3.4.2 前処理：識別子の正規化

ランダム化された長大な識別子を，その種別に応じて `var1`，`class1`，`method1` のような普遍的かつ短い名前へ変換する．コンテキストの圧縮による推論能力の低下を抑えるためである．

3.4.3 LLM エージェントへの指示

解析者は次の指示を LLM エージェントへ与える．

- ファイルが大きい場合や数が多い場合に制御フローの解除精度が低下すること [8] を前提に，解除の順序と単位を自ら決めること
- 残った難読化を解除し，識別子を役割がわかる名前へ変更すること
- 解除後にビルドし，エラーが発生しないことを確認すること
- プレースホルダへの値の割り当ては解析者が行うため，自らは割り当てないこと
- ビルドした実行ファイルを実行しないこと

4. RQ の設定と実験の構成

4.1 RQ の設定と実験の対応

本研究では，次の 3 つの RQ に答えることを目的として実験を設計した．

RQ1 LLM オーケストレーションで生成した解除スクリプトが難読化を解除できるか．

RQ2 解除スクリプトと LLM の推論によってマルウェアの難読化を解除できるか．

RQ3 マルウェア解析において有益な知見がどれくらい得られるか．

これらを検証するため，実験 A（5 章，RQ1），実験 B（6 章，RQ2 と RQ3），実験 C（7 章，RQ2 と RQ3）を実施する．いずれの実験でも，構文的解除度・意味的解除度・コストの 3 つの側面から評価する．

4.2 難読化の設定

難読化ツールには ConfuserEx2 v1.6.0[3] と.NET Reactor 7.5.0.0[2] を用いる．1 章で述べたとおり，実運用の.NET マルウェアで使用される難読化ツールの上位 2 つであるためである．いずれも実験 A と実験 B で用いる．

難読化手法ごとの解除性能を評価するため，難読化手法の組み合わせを次の 5 通り設定した．

組み合わせ α 識別子の難読化と制御フローの難読化

組み合わせ β 識別子の難読化と定数の難読化

組み合わせ γ 識別子の難読化と参照プロキシによる難読化

組み合わせ δ 識別子の難読化とリソースの難読化

組み合わせ ε 上記の 5 手法すべての複合

各組み合わせに対して両ツールで指定したオプションは

© 2026 Information Processing Society of Japan

表 1 解除スクリプトによる難読化解除の結果（実験 A）
Table 1 Results of deobfuscation by the generated scripts (Experiment A).

検体	難読化ツール	α	β	γ（TPR/TNR）	δ
LimeLogger	ConfuserEx2	1.02	1.00	1.00 / 0.17	—
	.NET Reactor	1.00	1.00	0.63 / 0.96	—
hidden-tear	ConfuserEx2	0.90	1.00	1.00 / 0.95	1.00
	.NET Reactor	0.84	1.00	0.38 / 0.97	1.00
REGFuck	ConfuserEx2	0.63	1.00	1.00 / 0.73	1.00
	.NET Reactor	0.56	1.00	1.00 / 0.97	1.00
Chaos	ConfuserEx2	0.42	1.00	1.00 / 0.02	—
	.NET Reactor	0.73	1.00	0.35 / 0.98	—
AsyncRAT	ConfuserEx2	0.49	1.00	0.47 / 0.50	—
	.NET Reactor	0.95	1.00	0.24 / 0.98	—

α：goto 残存割合，β：定数復元率 R，γ：参照プロキシ除去率 TPR/TNR，δ：リソース復元率 R.

付録 A.1.2 に示す．

すべての組み合わせに識別子の難読化を含めるのは，識別子が制御フローや定数の難読化を解除する手掛かりとなり，解除性能を正しく評価できなくなるためである．

4.3 LLM モデルとハーネスの選択とフォールバック

主に使用するモデルは Claude Opus 5 であり，ハーネスには Claude Code を利用する．ガードレールが発動して解除が中断した場合は，プロンプトを変更せず旧バージョンのモデルへ切り替えて解除を再開する．以下，この旧バージョンのモデルへの切り替えをフォールバックと呼ぶ．切り替えの順序は付録 A.1.3 に示す．

4.4 逆コンパイラの改造と設定変更

逆コンパイラには ILSpy v10.0.1.0[10] を用いる．参照プロキシや制御フローの難読化による逆コンパイルの失敗への対処と，LLM の推論の精度向上のための調整として，3 点の改造と設定変更を施した（付録 A.1.4）．

5. 実験 A：解除スクリプトによる難読化解除

5.1 実験の目的

LLM オーケストレーションで生成した解除スクリプトが難読化を解除できるかを明らかにする（RQ1）．本実験で評価するのは生成プロセス自体ではなく，生成された解除スクリプトの解除性能である．

5.2 実験で使用するマルウェア

vx-underground や GitHub でソースコードが公開・リークされているマルウェア（以下，ソース公開マルウェア）を母集団とし，5 検体を選定した．選定の基準は，行数とファイル数が小さいものから大きいものまで幅広く含まれ，かつマルウェアの種別が偏らないことである．行数は 201 行（2 ファイル）から 2,381 行（25 ファイル）まで，種別はキーロガー・ランサムウェア・ワイパー・RAT の 4 種別にわたる．ここでの行数は，いずれも難読化前の実行ファイルを逆コンパイルして得られた C#コードの行数である．

選定した 5 検体は，LimeLogger，hidden-tear，REGFuck（改造版），Chaos，AsyncRAT である（付録 A.3.1）．

5.3 難読化設定

4.2 節で定義した組み合わせのうち，組み合わせ α から組み合わせ δ を使用する．組み合わせ ε は複合難読化であるため，実験 B で評価する．難読化ツールは 4.2 節で述べた 2 種である．

5.4 評価指標

組み合わせごとに次の指標を用いる．算出方法の詳細はいずれも付録 A.4 に示す．

- α：**goto 残存割合**　制御フローの平坦化が生む多数の goto 文をどれだけ削減できたかを表す（0 に近いほどよい）．
- β：**定数復元率（Recall）**　オリジナル検体のソースコードに含まれる定数を正解とし，暗号化された定数をどれだけ正確に復号できたかを表す（1.000 が完全復元）．
- γ：**参照プロキシ除去率（TPR/TNR）**　参照プロキシをどれだけ正確に削除できたか（TPR），プロキシ以外のメソッド定義をどれだけ正確に残せたか（TNR）を表す．
- δ：**リソース復元率（Recall）**　オリジナル検体のリソースをどれだけ正確に復号できたかを表す（1.000 が完全復元）．

5.5 実験結果

難読化ツール 2 種と難読化手法 4 種の計 8 組について LLM オーケストレーションを実行し，解除スクリプトを生成した．生成時間は一手法あたり 1.5 時間から 2 時間程度であったが，一度生成すれば再利用できる．

難読化した 5 検体へ難読化手法ごとの解除スクリプトを適用した結果を**表 1** に示す．解除スクリプトによる解除は同じ入力に対して常に同じ結果を返し，試行間の揺らぎが生じないため，各条件の試行回数は 1 回である．

制御フローの難読化（組み合わせ α）では，全条件で goto 文が残存し，残存割合は 0.420 から 1.020 であった．削減できた条件でも削減率は 5%から 58%にとどまり，LimeLogger では削減できなかった．

定数の難読化（組み合わせ β）では，10 通り（2 ツール×5 検体）すべてで復元率 1.000 を達成した．

参照プロキシによる難読化（組み合わせ γ）は両ツールとも解除に失敗したが，失敗の様態は逆であった．ConfuserEx2 では TPR が 5 検体中 4 検体で 1.000 と高い一方，TNR は 0.020 から 0.950 と低くばらついており，プロキシ以外のメソッド定義を巻き込んで削除した．Chaos と LimeLogger ではその 8 割以上が失われた．.NET Reactor

© 2026 Information Processing Society of Japan

では TNR が 0.960 から 0.980 と安定する一方，TPR は 0.240 から 1.000 で 5 検体中 4 検体が 1.000 に届かず，プロキシを削除しきれなかった．

リソースの難読化（組み合わせ δ）では，リソースを持つ 2 検体（hidden-tear，REGFuck）で両ツールとも復元率 1.000 を達成した．残る 3 検体にはリソースが存在しないため，リソース復元率の評価は行っていない．

制御フローの難読化と参照プロキシによる難読化の解除が正しく行われなかった原因と対策は 8 章で述べる．

5.6 RQ1 への回答

定数とリソースの難読化は，LLM オーケストレーションで生成した解除スクリプトで解除できる．評価したすべての条件で復元率は 1.000 であった．

一方，制御フローの難読化と参照プロキシによる難読化は，スクリプトでは正確に解除できなかった．したがって，この 2 つの手法は LLM の推論で解除する必要がある．

6. 実験 B：ソース公開マルウェアの難読化解除

6.1 実験の目的

ソース公開マルウェアの難読化を解除スクリプトと LLM の推論で解除できるか（RQ2），解除したマルウェアから有益な知見がどれくらい得られるか（RQ3）を明らかにする．

6.2 実験で使用するマルウェアと難読化設定

検体は実験 A と同じ 5 検体である．難読化は組み合わせ ε を使用する．

6.3 評価指標

次の 3 つの側面から評価する．各指標の算出手順は付録 A.4 に示す．LLM の推論による揺らぎを考慮して，解除の試行は 3 回行う．

構文的解除度 再コンパイル可能率，再実行可能率，複雑度軽減指数 [11] を用いる．複雑度軽減指数は，ハルステッド・メトリクスに基づくプログラム長を用い，難読化が加えた複雑さをどれだけ除去できたかを表す．1.0 は元の複雑度まで完全に戻せたこと，0.0 は全く解除されていないことを意味する．

意味的解除度 API 列類似度，IoC 復元 F1，ATT&CK 復元 F1，定数復元率，リソース復元率を用いる．API 列類似度（相対忠実度）は，サンドボックスで記録された API 呼び出し履歴がオリジナル検体とどれくらい類似しているかを表す．IoC 復元 F1 と ATT&CK 復元 F1 は，CAPEv2 サンドボックスのレポートファイル（以下，CAPEv2 のレポート）に記録された IoC と MITRE ATT&CK Technique ID を，オリジナル検体と同じだけ再現できたかを表す F1 値である．定数復元率とリソース復元率は 5 章で定義したものである．

コスト 解除時間を用いる．LLM が解除の手順を計画する時間と，推論による解除の時間の合計である．

6.4 実験結果

実験結果を**表 2** に示す．API 列類似度は 8 試行が 0.37 から 0.87，22 試行が 0.952 から 0.997 と二分されたため，谷にあたる 0.900 を解除成功の境界とした．表には解除に成功した試行で得られた指標の平均値を記載している．

構文的解除度については，再コンパイル可能率と再実行可能率が 30 試行すべてで 100%であった．成功試行での複雑度軽減指数は ConfuserEx2 が 0.955 から 1.000，.NET Reactor が 0.787 から 0.977 であり，難読化による複雑さの大部分を除去してオリジナルに近い複雑度に戻せている．

意味的解除度については，ATT&CK 復元 F1 と定数復元率が成功した 22 試行すべてで 1.000 であった．IoC 復元 F1 は 0.909 から 1.000 で，22 試行中 16 試行が 0.950 以上であった．API 列類似度は 0.952 から 0.997 であった．解除時間は成功試行あたり 21 分から 98 分であった．

全 30 試行のうち 22 試行（73.33%）が成功し，難読化ツール別では ConfuserEx2 が 13/15（86.67%），.NET Reactor が 9/15（60.00%）であった．

ConfuserEx2 では hidden-tear，Chaos，AsyncRAT で各 1 回フォールバックが発生したが，いずれも解除を完了した．一方，.NET Reactor の REGFuck の 2 回目，AsyncRAT の 2 回目と 3 回目の試行では，フォールバックせず解除が完了したにもかかわらず，悪性機能に相当する処理のみが失われた．これら 3 試行は API 列類似度が低い側の 8 試行に含まれる．高い値が得られるまで解除を繰り返せば，誤った解除結果を分析することを避けられる．なお，実運用では難読化検体の API 列類似度を基準とすればよい．

6.5 RQ への回答

RQ2 については，実験 A で解除できなかった制御フローの難読化と参照プロキシによる難読化を LLM の推論で解除できた．また，失敗しても再試行すれば解除結果が得られるため，難読化ツールごとに手順を再構築する必要はない．

RQ3 については，再コンパイルと再実行が可能でオリジナルに近い複雑度のコードが得られ，ATT&CK と定数とリソースは完全に，IoC も 0.949 以上で復元できた．ただし失敗試行では悪性機能のみが失われることがあり，API 列類似度による成否判定と再試行が前提となる．

7. 実験 C：ソース未公開マルウェアの解除

7.1 実験の目的

ソース未公開マルウェアの難読化を解除スクリプトと LLM の推論で解除できるか（RQ2），そこから有益な知見がどれくらい得られるか（RQ3）を明らかにする．

© 2026 Information Processing Society of Japan

表 2 ソース公開マルウェアの難読化解除の結果（実験 B）
Table 2 Results of deobfuscating open-source malware (Experiment B).

難読化ツール	検体	構文的解除度 複雑度軽減指数	意味的解除度 API 列類似度	IoC 復元 F1	ATT&CK 復元 F1	定数復元率	リソース復元率	コスト 解除時間（分）	成功回数
ConfuserEx2	LimeLogger	0.991	0.970	0.958	1.000	1.000	—	21.0	3
	hidden-tear	0.987	0.997	1.000	1.000	1.000	1.000	38.5	2
	REGFuck	1.000	0.994	0.949	1.000	1.000	1.000	22.8	3
	Chaos	0.989	0.993	0.966	1.000	1.000	—	74.0	2
	AsyncRAT	0.955	0.952	1.000	1.000	1.000	—	91.0	3
.NET Reactor	LimeLogger	0.823	0.979	0.983	1.000	1.000	—	44.0	2
	hidden-tear	0.958	0.988	0.989	1.000	1.000	1.000	40.5	2
	REGFuck	0.803	0.988	0.954	1.000	1.000	1.000	43.5	2
	Chaos	0.787	0.994	0.975	1.000	1.000	—	71.5	2
	AsyncRAT	0.977	0.989	0.973	1.000	1.000	—	98.0	1

表 3 ソース未公開マルウェアの難読化解除の結果（実験 C）
Table 3 Results of deobfuscating closed-source malware (Experiment C).

検体	意味的解除度 類似度	IoC	ATT&CK	コスト 時間	コード規模 行数	実質
検体 1	1.000	0.974	1.000	47m	1,349	1,297
検体 2	0.996	1.000	1.000	18m	10,541	253
検体 3	1.000	0.987	1.000	35m	14,958	1,415
検体 4	0.993	0.982	1.000	29m	9,742	406

類似度：API 列類似度，IoC：IoC 復元 F1，ATT&CK：ATT&CK 復元 F1，行数（実質）：全行数（実質コード行数）．

7.2 実験で使用するマルウェア

MalwareBazaar および VirusShare で収集した，ソースコードが公開されていない.NET マルウェア（以下，ソース未公開マルウェア）4 検体を対象とする．これらは実環境で観測された検体であり，マルウェア開発者が難読化を施している．

選定は次の手順で行った．Detect It Easy により ConfuserEx 系と判定された検体のうち，本研究の対象外とする難読化（パッキング，仮想化，耐タンパーなど）を含む検体，および複数の難読化ツールで難読化された検体を除く．残った検体群から無作為に 4 検体を選んだ．

適用されていた難読化の組み合わせは，検体 1 が組み合わせ α，検体 2 が組み合わせ ε から制御フローの難読化を除いた構成，検体 3 と検体 4 が組み合わせ ε である（付録 A.3.2）．

7.3 評価指標

難読化前のオリジナル検体が存在しないため，難読化検体を正解とし，解除された検体がその挙動を再現できているかを評価する．複雑度軽減指数と，定数およびリソースの復元率は，オリジナル検体のソースコードがないため評価しない．それ以外の評価指標は実験 B と同じである．

7.4 実験結果

実験の結果を**表 3** に示す．4 検体すべてで難読化を解除でき，試行回数 1 回で全検体が成功した．意味的解除度は，API 列類似度が 0.993 から 1.000，ATT&CK 復元 F1 が全検体で 1.000，IoC 復元 F1 が 0.974 から 1.000 であった．

検体 2 では，リソースの難読化に対する解除スクリプトの適用に失敗した．原因は，実験 A で使用した ConfuserEx2 と，検体 2 が使用したフォーク元の ConfuserEx との間で，リソースの展開後のサイズの記録フォーマットが異なることであった（詳細は付録 A.5）．しかし，最終的にこのリソースの難読化は 3.4 節の手順どおり第 3 段階の LLM の推論によって解除できた．

表中の全行数は，難読化された検体を逆コンパイルして得られた C#コードのサイズであり，実験 A と実験 B とは基準が異なる．実質行数は，定数やリソースに関連したコードを除外した行数，すなわち第 3 段階で LLM の推論の対象となるコード行数である．解除時間は，全行数ではなく実質行数に影響していることがわかる．定数とリソースの難読化を含む検体 2 から検体 4 までは，第 2 段階のスクリプトによって LLM の推論の対象が全行数の 2%から 10%に縮小した．

7.5 RQ への回答

RQ2 については，4 検体のいずれも提案手法で解除できた．うち 1 件は解除スクリプトが対応しないフォークであったが，LLM の推論がその不足を補った．フォークの同定が必須ではない可能性を示す結果である．

RQ3 については，ATT&CK と IoC を高い精度で再現でき，検体の規模が 1 万行前後であっても実験 B の 5 検体と同程度以下のコストで解除できた．ただしオリジナルのソースコードという正解が存在せず，定数とリソースの復元率は評価できない．そのため，この結果は適用可能性を補足的に示すものである．

8. 考察

8.1 制御フロー難読化の解除失敗の原因と対策

解除スクリプトは，難読化ツールを固定すれば解除対象

© 2026 Information Processing Society of Japan

の構文も一定になることを前提としている．しかし実際には，制御フローの平坦化が逆コンパイル後の C#で表現される構文的な形状は，同一のツール・同一のオプションであっても一定しない．同一検体の中でもメソッドによって異なる．形状が変わるのは，難読化前のコード構造（分岐やループの入れ子，例外処理の有無）に応じて平坦化後の CIL が変わり，さらに ILSpy が元のループや分岐をどこまで復元できるかがその CIL のコードパターンに依存するためである．

一方，解除スクリプトがカバーできるのは，生成時に用いた検証用サンプル 10 個に現れた構文パターンに限られる．そこから外れた形状のディスパッチャはカバーされずに残るため，制御フローの難読化を網羅的に解除できない．

対策として，C#と併せて CIL を参照させることが考えられる．逆コンパイル後の C#で未知の構文パターンが現れても，CIL の段階では平坦化が switch 命令と状態変数の更新として一定の形で残る．形状の違いに依存しない，網羅性のある解除スクリプトを生成できる可能性がある．

8.2 参照プロキシ難読化の解除失敗の原因と対策

参照プロキシはメソッド呼び出しを隠蔽するため，どのメソッド定義がプロキシで，どのメソッド定義がどこから使われているかを，逆コンパイル後の C#から判定できない．さらに，いずれの組み合わせにも識別子の難読化を含めているため，メソッド名も判定の手掛かりにならない．

その結果，解除の成否は，プロキシと確定できないメソッド定義を残す側と削除する側のどちらを優先するかに帰着する．ここでプロキシと確定できないメソッド定義とは，どこからも呼ばれない関数や，元のコードに存在するラッパーなどである．残せば削除漏れが生じ，削除すれば必要なメソッドの定義まで失われる．いずれにせよ解析に使えるコードは得られない．実際に生成された解除スクリプトは，.NET Reactor では残す側を，ConfuserEx2 では削除する側を優先しており，削除漏れと過剰削除という相反する結果として現れた．

対策は制御フローの場合と同じく，CIL の併用である．呼び出し先と呼び出し元の関係は CIL のメタデータに残るため，これを併せて参照させればプロキシかどうかを判定できる可能性がある．

9. 結論

本研究では，LLM オーケストレーションで解除スクリプトを生成し，スクリプトで解ける難読化を先に解除して，残った難読化を LLM の推論に委ねる 3 段階のフレームワークを提案した．ConfuserEx2 と.NET Reactor で難読化したソース公開マルウェア 5 検体，および脅威アクターが難読化を施したソース未公開マルウェア 4 検体を用いて評価した．

定数とリソースの難読化は解除スクリプトのみで完全に復元でき，制御フローの難読化と参照プロキシによる難読化は LLM の推論を要した．スクリプトで自動化できる範囲と LLM の推論に委ねる範囲の境界がここにある．5 つの手法を複合した難読化に対しては，ConfuserEx2 と.NET Reactor のいずれでも，再コンパイルと再実行が可能なコードを同一の手順で復元できた．解除後のコードから ATT&CK と定数とリソースを完全に，IoC を高い精度で復元でき，1 検体あたり 18 分から 1 時間 38 分で解析可能な状態へ戻せた．解除の成否は API 列類似度で判定できるため，再試行と組み合わせれば，試行ごとに揺らぐ LLM の推論を実用的な解析手順へ組み込める．

次の課題は 2 つある．逆コンパイル後の C#に加えて CIL とメタデータを併用し，制御フローの難読化と参照プロキシによる難読化もスクリプトで解除して自動化の範囲を広げることである．そして，第 1 段階に限られている LLM オーケストレーションを，第 3 段階まで拡張し，解析者の介入を最小化することである．

謝辞 本研究は JSPS 科研費 24K14954 の助成を受けたものである．

参考文献

[1] UnpacMe: 2023 malware trends (2024).
[2] Eziriz: .NET Reactor, `https://www.eziriz.com/dotnet_reactor.htm`.
[3] Martin Karing: ConfuserEx, `https://github.com/mkaring/ConfuserEx`.
[4] Tkachenko, A., Suskevic, D. and Adolphi, B.: Deconstructing obfuscation: a four-dimensional framework for evaluating large language models assembly code deobfuscation capabilities, *arXiv:2505.19887* (2025).
[5] Chen, G., Jin, X. and Lin, Z.: JsDeObsBench: measuring and benchmarking LLMs for JavaScript deobfuscation, *Proc. ACM CCS*, pp. 36–50 (2025).
[6] Mariano, B., Wang, Z., Pailoor, S., Collberg, C. and Dillig, I.: Control-flow deobfuscation using trace-informed compositional program synthesis, *Proc. ACM Program. Lang.*, Vol. 8, No. OOPSLA2, pp. 2211–2241 (2024).
[7] Feng, R. and Saha, S.: Can LLMs recover program semantics? A systematic evaluation with symbolic execution, *arXiv:2511.19130* (2025).
[8] 栗原北斗，河野龍之介，岡本 剛：LLM による.NET マルウェアの難読化解除：概念実証に向けた基礎的検証，情報処理学会研究報告， Vol. 2026-CSEC-112, No. 46, pp. 1–8 (2026).
[9] Xu, H., Zhou, Y., Ming, J. and Lyu, M.: Layered obfuscation: a taxonomy of software obfuscation techniques for layered security, *Cybersecurity*, Vol. 3, No. 1, p. 9 (2020).
[10] ICSharpCode: ILSpy - .NET decompiler, `https://github.com/icsharpcode/ILSpy`.
[11] Beste, D., Menguy, G., Hajipour, H., Fritz, M., Cinà, A. E., Bardin, S., Holz, T., Eisenhofer, T. and Schönherr, L.: Exploring the potential of LLMs for code deobfuscation, *Proc. DIMVA*, pp. 267–286 (2025).

© 2026 Information Processing Society of Japan

10. 研究倫理

10.1 研究の目的と正当性

本研究は，難読化されたマルウェアの解析工程を支援し，インシデントレスポンスや脅威解析における初動判断の迅速化と，従来は解析者の手作業や経験に依存していた工程の工数削減に寄与する．

10.2 対象範囲とライセンス・規約の遵守

本研究の評価対象は，ソース公開マルウェア，実環境で観測されたソース未公開マルウェア，および実験用に作成した無害なコンソールアプリケーションに限定している．

難読化ツールのライセンスについては次のとおり確認した．ConfuserEx2 は MIT ライセンス[*1]の下で公開されており，利用や成果物の解析を制限するエンドユーザー使用許諾契約（EULA）は存在しない．商用の.NET Reactor については公式の EULA[*2]を確認したところ，同 EULA が禁止する逆コンパイル・逆アセンブル・リバースエンジニアリングは「.NET Reactor 製品そのもの」を対象としており，同ツールで難読化されたソフトウェアの解析を制限するものではなかった．本研究は製品自体の解析・改変・再配布を行わないため，同 EULA に抵触しないと解釈できる．

LLM の利用に関する規約の遵守と安全対策の尊重については，次の 2 点を満たしている．

LLM へ送信するデータ　LLM へ入力するのは，難読化された検体を逆コンパイルして得た C#コードと，解除を指示するプロンプトのみであり，検体の実行ファイル自体は送信しない．また，定数はプレースホルダへ置き換えた状態で入力するため（3.4 節），C2 サーバのアドレスや暗号鍵などの IoC を外部サービスへ送信することはない．

安全機構を迂回していないこと　ジェイルブレイクなど安全機構を迂回する手法は用いず，提供元の許諾範囲内での利用に限定している．ガードレールが発動して解除が中断した場合は，プロンプトを変更せず旧バージョンのモデルへ切り替えて解除を再開した．本研究が行ったのは利用するモデルの選択のみであり，いずれの試行も各モデルがその安全方針の範囲内で応答した結果のみを用いている．

10.3 検体の取り扱いと解析環境の安全性

検体の動的解析および解除後の検体の実行は，外部ネットワークから論理的に隔離された，安全な解析用サンドボックス環境内において実施した．

本研究は再コンパイル可能率や API 列類似度を評価指標とするため，解除したソースコードをビルドして実行ファイルを生成し，これを実行する（3.4 節）．これは動作するマルウェアを再生成する行為を含む．この点を認識し，生成した実行ファイルおよび解除後のソースコードの実行と保管を，上記の隔離環境内に限定した．

実験に使用した検体は，vx-underground や MalwareBazaar など研究目的のサービスより，各サイトの提供方針に従って取得した．

10.4 デュアルユースのリスクへの配慮

本研究が扱う難読化解除の技術は，適用される領域によって防御と攻撃の役割が逆転する．

マルウェア解析の文脈　難読化は攻撃者側（マルウェア開発者）による解析妨害の手段であり，その解除は防御側（解析者）が脅威を解明するための正当な活動である．

チート・著作権保護の文脈　難読化は防御側（ソフトウェア開発者）がオンラインゲームにおけるプレイヤー間の公平性やソフトウェアの知的財産を維持するための保護手段であり，その解除は攻撃側（チーターやクラッカー）による悪用行為に直結する．

マルウェア再開発の文脈　脅威アクターが提案手法を悪用し，MalwareBazaar などで配布されているマルウェア検体の難読化を解除して，それを改造・再利用する可能性が考えられる．

このリスクを軽減するため，LLM へ入力した生のプロンプト，生成された解除スクリプトのコード，および解除後のソースコードと実行ファイルは公開しない．一方，手法の設計方針，3 段階のフレームワークの構成，プロンプトに含めた要件，および評価結果（3 章および 5 章から 7 章）は公開する．これにより，防御側が手法を理解し再構成するために必要な知見は共有しつつ，そのまま実行できる解除ツールと解除済みのマルウェアを脅威アクターへ提供することは避ける．

本手法が正当な保護技術の回避に転用されるリスクは否定できない．それでも，防御側において解析手法の知見を共有し，LLM を用いた難読化解除の限界と有効な適用範囲を明らかにすることは，既存の保護手法の堅牢性を再検証する一助となり，セキュリティ技術の向上に寄与すると考えられる．

付　　録

A.1 実験環境の詳細

A.1.1 ConfuserEx2 のレイヤー別機能分類

2.1 節で述べたレイヤー分類に対応する ConfuserEx2 の難読化機能を**表 A·1** に示す．

[*1] https://github.com/mkaring/ConfuserEx/blob/master/LICENSE.md（参照 2026-08-21）
[*2] https://www.eziriz.com/help/license_agreement/index.html（参照 2026-08-21）

© 2026 Information Processing Society of Japan

表 A·1　ConfuserEx2 の難読化機能のレイヤー別分類
Table A·1　Obfuscation features of ConfuserEx2 classified by layer.

レイヤー	難読化手法	ConfuserEx2 における名称
コード要素	識別子の難読化	Name Protection
	制御フローの難読化	Control Flow Protection
	定数の難読化	Constants Protection
	参照プロキシによる難読化	Reference Proxy Protection
ソフトウェアコンポーネント	改ざん防止	Anti Tamper Protection
	逆コンパイル妨害	Anti IL Dasm Protection
	デバッグ妨害	Anti Debug Protection
	ダンプ妨害	Anti Dump Protection
	圧縮	Compressing Packer
	仮想化	—
クロスコンポーネント	リソースの難読化	Resources Protection

A.1.2　難読化の組み合わせとツール別のオプション

4.2 節で定義した組み合わせ α から ε に対して，両ツールで指定したオプションを**表 A·2** に示す．

A.1.3　LLM モデルのフォールバック

主に使用するモデルは Claude Opus 5 である（4.3 節）．安全ガードレールが発動して解除が中断した場合は，Claude Opus 4.8，Claude Opus 4.7，Claude Opus 4.5，Claude Sonnet 4.6 の順にモデルを切り替えて再実行する．Claude Opus 5 から Claude Opus 4.8 への切り替えは自動で発生し，それ以降は手動で行う．切り替えるのはモデルのみであり，プロンプトは変更しない．

A.1.4　逆コンパイラの改造と設定変更

4.4 節で述べた ILSpy v10.0.1.0 に対して，次の 3 点を変更した．

配列初期化コードの逆コンパイル処理の改造　参照プロキシによる難読化によって配列の初期化コードのパターンが破壊されると逆コンパイルされないため，パターンが破壊されても逆コンパイルされるように改造した．

非同期メソッドの逆コンパイル無効化の設定　制御フローの難読化によって非同期メソッドの CIL パターンが破壊され，逆コンパイルに失敗するため，この機能を設定で無効化した．

クラスノードごとの逆コンパイル機能の追加　標準では複数の異なるクラスが 1 つのファイルへ集約されてファイルサイズが大きくなり，LLM の推論による解除の精度が落ちることがあるため，クラスノードごとに逆コンパイルしてファイルへエクスポートする機能を追加した．

A.2　メタプロンプトが生成するサブエージェント

3.2 節のメタプロンプトから生成される 8 つのサブエージェントの役割と主な出力を**表 A·3** に示す．LLM オーケストレーションの単位は難読化ツール 1 つと難読化手法 1 つの組であり，1 回の実行で対象とする難読化手法（以下，難読化 X）は 1 つである．

A.3　実験で使用したマルウェアの詳細

A.3.1　ソース公開マルウェア（実験 A・実験 B）

5 章で挙げた 5 検体の種別・行数・ファイル数・動作を**表 A·4** に示す．表中の行数は，いずれも難読化前の実行ファイルを逆コンパイルして得られた C#コードのサイズであり，人間が作成したソースコードを対象にしない．

A.3.2　ソース未公開マルウェア（実験 C）

7 章で対象とした 4 検体の詳細を**表 A·5** に示す．

A.4　各指標の算出方法

5 章から 7 章の各指標について，算出に用いた定義と手順を示す．サンドボックスは CAPEv2 を用い，オリジナル検体と解除検体をそれぞれ 5 回実行したレポートを入力とする．

A.4.1　goto 残存割合

制御フローの難読化では制御フローの平坦化が行われ，多数の goto 文が生じてコードの読解を難しくする．goto 残存割合は，解除後の C#コードに含まれる goto 文の行数を，難読化後の C#コードに含まれる goto 文の行数で割った値とする．0 に近いほど制御フローの平坦化を解除できたことを意味する．オリジナル検体のソースコードに goto 文は存在しない．

A.4.2　再コンパイル可能率・再実行可能率

全試行数を N_{Trial}，再コンパイルに成功した試行数を N_{Build}，再実行に成功した試行数を N_{Run} とする．

再コンパイル可能率 R_{Build} は，解除後の C#コードを

© 2026 Information Processing Society of Japan

表 A·2　難読化の組み合わせとツール別のオプション
Table A·2　Obfuscation combinations and the corresponding options for each tool.

組み合わせ	難読化手法	ConfuserEx2	.NET Reactor
α	識別子＋制御フロー	`rename`, `ctrl flow`	`Obfuscation`, `Control_Flow_Obfuscation`
β	識別子＋定数	`rename`, `constants`	`Obfuscation`, `String_Encryption`
γ	識別子＋参照プロキシ	`rename`, `ref proxy`	`Obfuscation`, `Hide_Method_Calls`
δ	識別子＋リソース	`rename`, `resources`	`Obfuscation`, `Resource_Encryption_And_Compression`
ε	上記すべて	上記のオプションすべて	上記のオプションすべて

表 A·3　メタプロンプトが生成する 8 つのサブエージェント
Table A·3　The eight subagents generated from the metaprompt.

サブエージェント	役割	主な出力
Orchestrator	全体の進行管理と環境の事前検証．実作業は行わず，各段の成果物を検証して次段へ進める判断のみを行う	環境検証の結果，各段の進行状況
Sample Generator	難読化 X が適用されやすい.NET Framework のコンソールアプリケーションを 10 個生成してビルドする	検証用サンプルのソースと実行ファイル，決定的な標準出力と終了コード（解除の正解）
Obfuscation Runner	解析者が指定したツールとオプションで 10 個を難読化する	難読化済み実行ファイル，難読化前後で実行結果が一致することの確認
Decompiler	難読化前後の実行ファイルを逆コンパイルする	難読化前後の C#ソースと差分，解除前の再コンパイル可否（責任範囲の切り分けに用いる）
Pattern Analysis	難読化前後の C#を突き合わせ，難読化 X の痕跡をパターンとして分類する	パターン識別子付きのカタログ（解除スクリプトの実装仕様）
Deobfuscator	カタログに基づき難読化 X を解除する解除スクリプト（Python）を実装する．引数は対象フォルダのパスのみ	解除スクリプト，復元コード，復元したものの一覧の JSON
Evaluation	10 個の難読化済み検証用サンプルへ解除スクリプトを適用し，復元率・実行結果の一致・可読性の改善を評価する	サンプルごとの評価結果と集計，失敗事例の一覧
Iteration & Report	評価結果と失敗事例をもとに解除スクリプトを改善する（メタプロンプトでは最低 3 サイクルを指示）	各サイクルの改善差分とテスト，報告

C#コンパイラでビルドし，エラーなくビルドを完了した場合を成功，コンパイルエラーが発生した場合を失敗として，次式で求める．

$$R_{\mathrm{Build}} = \frac{N_{\mathrm{Build}}}{N_{\mathrm{Trial}}} \tag{A.1}$$

再実行可能率 R_{Run} は，再コンパイルに成功した試行を分母とし，コンパイルした実行ファイルを実行して，実行時エラーを出さずに動作した場合を成功，実行時エラーが出て実際の動作を行わなかった場合を失敗として，次式で求める．

$$R_{\mathrm{Run}} = \frac{N_{\mathrm{Run}}}{N_{\mathrm{Build}}} \tag{A.2}$$

いずれも 1.000 であれば，解除後のコードが静的解析だけでなく動的解析にも供せることを意味する．

A.4.3　複雑度軽減指数

Beste ら [11] の定義に従い，ハルステッド・メトリクスに基づくプログラム長を C とする．難読化前を C_{Orig}，難読化後を C_{Obf}，解除後を C_{Deobf} として，複雑度軽減指数 P_{Deobf} を次式で求める．

$$P_{\mathrm{Deobf}} = 1 - \frac{C_{\mathrm{Deobf}} - C_{\mathrm{Orig}}}{C_{\mathrm{Obf}} - C_{\mathrm{Orig}}} \tag{A.3}$$

値の意味は次のとおりである．

$P_{\mathrm{Deobf}} > 1.0$　オリジナルより単純な表現になった．

$P_{\mathrm{Deobf}} = 1.0$　難読化が加えた複雑さを完全に除去して元の複雑度まで戻せた．

$P_{\mathrm{Deobf}} = 0.0$　全く解除されていない（難読化後と同じ複雑度）．

$P_{\mathrm{Deobf}} < 0.0$　難読化後よりさらに複雑になった（解除の失敗）．

A.4.4　API 列類似度

CAPEv2 のレポートの `behavior.processes[]` 以下の `calls[]` から，API 名を時刻順に並べた列を比較単位とする．2 つの API 列の類似度 Sim は，Jaccard 係数 S_{Jaccard}，コサイン類似度 S_{Cosine}，3-gram 類似度 S_{3gram} の平均とする．S_{Jaccard} は出現した API 集合の重なり，S_{Cosine} は API 呼び出し頻度ベクトルのコサイン類似度，S_{3gram} は連続する 3 つの API 呼び出しの組の集合に対する Jaccard 係数である．

© 2026 Information Processing Society of Japan

表 A·4 実験 A と実験 B で使用したソース公開マルウェア 5 検体
Table A·4 The five open-source malware samples used in Experiments A and B.

検体	種別	行数	ファイル数	動作
LimeLogger	キーロガー	201	2	キー入力を記録して外部に送信する.
hidden-tear	ランサムウェア	383	5	主要 12 フォルダを再帰的に走査して暗号化し，デスクトップにランサムノートを生成する.
REGFuck	ワイパー	666	6	レジストリに含まれるエントリーをランダムに削除する．ワイプする前に同意を求める機能がサンドボックスによる自動解析の障害となるため，この機能を除いた改造版を用いた.
Chaos	ランサムウェア	821	3	すべてのフォルダに含まれるファイルを暗号化し，各フォルダにランサムノートを生成する（ただし Windows フォルダなどのシステムフォルダとプログラムファイルフォルダは除く）.
AsyncRAT	RAT	2,381	25	AsyncRAT の C2 サーバに接続し，サーバの指示を待ち受ける.

表 A·5 実験 C で対象としたソース未公開マルウェア 4 検体
Table A·5 The four closed-source malware samples used in Experiment C.

検体	収集元	MD5	初見日	Microsoft の検出名	難読化の組み合わせ
1	VirusShare	`336d76f9227a7cbdf8d0e5685f3411c0`	2021-10-27	Backdoor:Win32/Bladabindi!ml	α
2	VirusShare	`5d554caaf210664ea715b3e07396cae2`	2018-01-25	Backdoor:Win32/Bladabindi!ml	ε から制御フローを除いた構成
3	MalwareBazaar	`0e9ae681b15e666fd020b7f2ffa17594`	2025-07-28	Trojan:MSIL/Heracles.AG!MTB	ε
4	MalwareBazaar	`a77431e2d2b1d029280cf0a47b113105`	2022-11-28	Trojan:Win32/Leonem	ε

$$\mathrm{Sim} = \frac{S_{\mathrm{Jaccard}} + S_{\mathrm{Cosine}} + S_{\mathrm{3gram}}}{3} \tag{A.4}$$

同一検体でも実行ごとにトレースが変動するため，類似度は 1.0 に達しない．そこでオリジナル検体 5 実行の全ペア 10 組の平均 μ を到達可能な上限とする．解除後検体 5 実行とオリジナル検体 5 実行の全クロスペア 25 組の平均を S_{Deobf} とし，これを μ で正規化した相対忠実度 RelFidelity を主指標とする.

$$\mathrm{RelFidelity} = \frac{S_{\mathrm{Deobf}}}{\mu} \tag{A.5}$$

本文の表では API 列類似度を小数で示す．0.900 は 6 章で定めた成否判定の境界である.

A.4.5 IoC 復元 F1

CAPEv2 のレポートから，ファイルの操作，レジストリの操作，ミューテックスなどの各カテゴリの IoC を抽出する．また，サンドボックス固有のデータや.NET アプリケーション共通のデータなどのノイズが含まれるため，事前にこれらを除外する．また，レポートに含まれるこれらのデータは同一の実行ファイルであっても実行のたびに変化するため，オリジナル検体の 5 回の実行で得られた共通のデータを正解 IoC 群とし，解除検体の 5 回の実行結果から得られた共通のデータを復元 IoC 群とする．それぞれの IoC 群に含まれる IoC を 1 つずつ照合して，正解 IoC と復元 IoC が一致したら True Positive（TP）として，正解 IoC にあり復元 IoC になければ False Negative（FN）として，正解 IoC になく復元 IoC にあれば False Positive（FP）として計上し，これらの混同行列から F1 値を計算する.

$$\mathrm{F1} = \frac{2 \cdot \mathrm{TP}}{2 \cdot \mathrm{TP} + \mathrm{FP} + \mathrm{FN}} \tag{A.6}$$

A.4.6 ATT&CK 復元 F1

CAPEv2 のレポートには CAPEv2 が検知したシグネチャに対応する ATT&CK Technique ID が付与されている．ATT&CK 復元 F1 はこの ATT&CK Technique ID を抽出して，IoC 復元 F1 と同様の要領で計算する.

A.4.7 参照プロキシ除去率（TPR/TNR）

難読化後のメソッド定義 1 件を 1 標本とし，プロキシを正しく削除した場合を TP，プロキシを誤って残こした場合を FN，プロキシ以外を正しく残こした場合を TN，プロキシ以外を誤って削除した場合を FP として数える．TPR はプロキシをどれだけ正確に削除できたか，TNR はプロキシ以外のメソッド定義をどれだけ正確に残せたかを表す.

© 2026 Information Processing Society of Japan

A.4.8 定数復元率・リソース復元率

定数の難読化の解除は，復号に成功して元の値がコードに現れるか，失敗して現れないかのいずれかであり，誤った値が復元されることはない．そこで，難読化前の逆コンパイル済み C#に含まれる定数を正解とし，そのうち解除後の C#に同じ値が現れたものを復元できた定数として数え，その割合を定数復元率とする．同じ値が複数回現れる場合は，出現した回数だけ数える．

$$\text{定数復元率} = \frac{\text{復元できた定数の個数}}{\text{難読化前の定数の個数}} \tag{A.7}$$

リソース復元率も，リソースデータを対象として同様に求める．

A.5 検体 2 における ConfuserEx のフォーク間の差異

実験 A で使用した ConfuserEx2 は，リソースの展開後のサイズを 4 バイトで記録する．一方，検体 2 が使用したフォーク元の ConfuserEx は 8 バイトで記録する方式であった．解除スクリプトは前者に合わせて実装されているため，サイズの読み取りと圧縮データの開始位置がともにずれ，リソースの復号に失敗した．このリソースの難読化は 3.4 節の手順どおり LLM の推論によって解除でき，フォーク元の仕様に合わせて解除していることを確認した．

© 2026 Information Processing Society of Japan